



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Отчёт о лабораторной работе № 3
по курсу «Разработка параллельных и распределенных
программ»

Студент: Ивенкова О. В.

Группа: ИУ9-52Б

Преподаватель: Царёв А. С.

Москва, 2025

Содержание

1	Постановка задачи	3
2	Практическая реализация	5
3	Сравнение вычислений	9
4	Характеристики устройства	10

1 Постановка задачи

Лабораторная работа №3: «Параллельная реализация решения системы линейных алгебраических уравнений с помощью OpenMP».

РЕШЕНИЕ СИСТЕМ ЛИНЕЙНЫХ АЛГЕБРАИЧЕСКИХ УРАВНЕНИЙ ИТЕРАЦИОННЫМИ МЕТОДАМИ

Пусть есть система из N линейных алгебраических уравнений в виде $Ax = b$, где A – матрица коэффициентов уравнений размером $N \times N$, b – вектор правых частей размером N , x – искомый вектор решений размером N . Решение системы уравнений итерационным методом состоит в выполнении следующих шагов.

1. Задается x_0 – произвольное начальное приближение решения (вектор с произвольными начальными значениями).
2. Приближение многократно улучшается с использованием формулы вида $x_{n+1} = f(x_n)$, где функция f определяется используемым методом.
3. Процесс продолжается, пока не выполнится условие $g(x_n) < \varepsilon$, где функция g определяется используемым методом, а величина ε задает требуемую точность.

Персональный вариант: Метод сопряженных градиентов.

Метод сопряженных градиентов

В методе сопряженных градиентов преобразование решения на каждом шаге задается следующими формулами:

$$\begin{aligned} r^0 &= b - Ax^0, \\ z^0 &= r^0, \\ \alpha^{n+1} &= \frac{(r^n, r^n)}{(Az^n, z^n)}, \\ x^{n+1} &= x^n + \alpha^{n+1} z^n, \\ r^{n+1} &= r^n - \alpha^{n+1} Az^n, \\ \beta^{n+1} &= \frac{(r^{n+1}, r^{n+1})}{(r^n, r^n)}, \\ z^{n+1} &= r^{n+1} + \beta^{n+1} z^n, \end{aligned} \tag{1}$$

где $(u, v) = \sum_{i=0}^{N-1} u_i v_i$. Метод сопряженных градиентов на каждой итерации содержит только одно умножение матрицы на вектор и, кроме того, сходится

быстрее, чем предыдущие два метода. Однако метод сопряженных градиентов работает только для симметричных матриц коэффициентов. Критерий завершения счета в методе сопряженных градиентов следующий:

$$\frac{\|r^n\|_2}{\|b\|_2} < \varepsilon, \quad \text{где} \quad \|u\|_2 = \sqrt{\sum_{i=0}^{N-1} u_i^2} \quad (2)$$

Подбирать значения параметра ε лучше всего исходя из задачи, начиная со значения 0.00001. Если, начиная с определенной итерации значение этого параметра перестаёт меняться, значит, можно выбрать в качестве критерия завершения счёта именно это установившееся значение.

ИСХОДНЫЕ ДАННЫЕ

Исходные данные для тестирования реализаций представленных методов и выполнения лабораторной работы можно взять следующие.

1. Модельная задача с заданным решением

Элементы главной диагонали матрицы A равны 2.0, остальные равны 1.0. Все элементы вектора b равны $N + 1$. В этом случае решением системы будет вектор, элементы которого равны 1.0. Начальные значения элементов вектора x можно взять равными 0.

2. Модельная задача с произвольным решением

Элементы главной диагонали A равны 2.0, остальные равны 1.0. Формируется вектор u , элементы которого заполняются произвольными значениями, например $u_i = \sin \frac{2\pi i}{N}$. Элементы вектора b получаются путем умножения матрицы A на вектор u . В этом случае решением системы будет вектор, равный вектору u . Начальные значения элементов вектора x можно взять равными 0.

ЗАДАНИЕ К ЛАБОРАТОРНОЙ РАБОТЕ

1. Написать программу, которая реализует итерационный алгоритм решения системы линейных алгебраических уравнений вида $Ax = b$ в соответствии с выбранным вариантом. Здесь A – матрица размером $N \times N$, x и b – векторы длины N . Тип элементов – double.

2. Программу распараллелить с помощью OpenMP с разрезанием матрицы A по строкам на близкие по размеру, возможно не одинаковые, части. Сосед-

ние строки матрицы должны располагаться в одном или в соседних OpenMP-процессах. Реализовать два варианта программы:

- 1: векторы x и b дублируются в каждом OpenMP-процессе,
- 2: векторы x и b разрезаются между OpenMP-процессами аналогично матрице A . (только для сдающих после срока)

Уделить внимание тому, чтобы при запуске программы на различном числе OpenMP-процессов решалась одна и та же задача (исходные данные заполнялись одинаковым образом).

3. Замерить время работы двух вариантов программы при использовании различного числа процессорных ядер: 1, 2, 4, 8, 16. Построить графики зависимости времени работы программы, ускорения и эффективности распараллеливания от числа используемых ядер. Исходные данные, параметры N и ε подобрать таким образом, чтобы решение задачи на одном ядре занимало не менее 30 секунд. Также параметр N разрешено подобрать таким образом, чтобы он нацело делился на 1, 2, 4, 8 и 16.

4. На основании полученных результатов сделать вывод о целесообразности использования одного или второго варианта программы.

2 Практическая реализация

Листинг 1: Код программы

```
1 #include <omp.h>
2 #include <vector>
3 #include <cmath>
4 #include <iostream>
5
6 using std::vector;
7
8 double dotProduct(const vector<double>& v1, const vector<double>& v2) {
9     double sum = 0.0;
10    int n = (int)v1.size();
11
12    #pragma omp parallel for reduction(+:sum)
13    for (int i = 0; i < (int)n; ++i)
14        sum += v1[i] * v2[i];
15
16    return sum;
```

```

17 }
18
19 double norm(const vector<double>& v) {
20     return std::sqrt(dotProduct(v, v));
21 }
22
23 vector<double> multiplyA(const vector<vector<double>>& A, const vector<
    double>& x) {
24     size_t n = A.size();
25     size_t m = A[0].size();
26     vector<double> res(n, 0.0);
27
28 #pragma omp parallel for
29     for (int i = 0; i < n; ++i) {
30         double s = 0.0;
31         for (int j = 0; j < m; ++j)
32             s += A[i][j] * x[j];
33         res[i] = s;
34     }
35     return res;
36 }
37
38 vector<double> solveCG(const vector<vector<double>>& A,
39     vector<double> x,
40     const vector<double>& b,
41     double eps)
42 {
43     size_t n = b.size();
44
45     vector<double> r(n), z(n);
46
47     vector<double> Ax = multiplyA(A, x);
48 #pragma omp parallel for
49     for (int i = 0; i < n; ++i) {
50         r[i] = b[i] - Ax[i];
51         z[i] = r[i];
52     }
53
54     double rr_old = dotProduct(r, r);
55     double norm_b = norm(b);
56     if (norm_b == 0.0) norm_b = 1.0;
57
58     int max_iter = (int)n;
59     int iter = 0;
60
61     for (; iter < max_iter; ++iter) {

```

```

62
63     vector<double> Az = multiplyA(A, z);
64     double Az_z = dotProduct(Az, z);
65
66     if (Az_z == 0.0) {
67         std::cerr << "INFO: Az_z == 0 on iter " << iter << "\n";
68         break;
69     }
70
71     double alpha = rr_old / Az_z;
72
73 #pragma omp parallel for
74     for (int i = 0; i < n; ++i) {
75         x[i] += alpha * z[i];
76         r[i] -= alpha * Az[i];
77     }
78
79     double rr_new = dotProduct(r, r);
80     double rel_res = std::sqrt(rr_new) / norm_b;
81
82     if (iter % 10 == 0 || iter < 5 || rel_res < eps) {
83         std::cout << "iter " << iter << " rel_res=" << rel_res
84             << " rr_new=" << rr_new << " Az_z=" << Az_z << "\n";
85     }
86
87     if (rel_res < eps) {
88         std::cout << "Converged at iter " << iter
89             << " rel_res=" << rel_res << "\n";
90         break;
91     }
92
93     double beta = rr_new / rr_old;
94
95 #pragma omp parallel for
96     for (int i = 0; i < n; ++i)
97         z[i] = r[i] + beta * z[i];
98
99     rr_old = rr_new;
100 }
101
102 std::cout << "Finished iterations: " << iter << "\n";
103
104 return x;
105 }
106
107 vector<vector<double>> generateMatrix(int n) {

```

```

108     vector<vector<double>>> A(n, vector<double>(n, 1.0));
109     for (int i = 0; i < n; ++i)
110         A[i][i] = 2.0;
111     return A;
112 }
113
114 int main() {
115
116     int n = 36864;
117     omp_set_num_threads(16);
118
119     vector<vector<double>>> A = generateMatrix(n);
120     vector<double> b(n, n + 1.0);
121     vector<double> x(n, 1000.0);
122
123     double eps = 1e-22;
124
125     double t_start = omp_get_wtime();
126     x = solveCG(A, x, b, eps);
127     double t_end = omp_get_wtime();
128
129     std::cout << "Total time = " << t_end - t_start << " sec\n";
130
131     double err = 0.0;
132 #pragma omp parallel for reduction(+:err)
133     for (int i = 0; i < n; ++i) {
134         double d = x[i] - 1.0;
135         err += d * d;
136     }
137     std::cout << "RMSE = " << std::sqrt(err / n) << "\n";
138
139     return 0;
140 }

```


3 Сравнение вычислений

Листинг 2: Вывод после запуска кода на 1, 2, 4, 8, 16 процессорных ядрах

```
1 --- 1 ---
2 iter 0 rel_res=1.07823e-09 rr_new=5.8244e-05 Az_z=1.84322e+24
3 iter 1 rel_res=7.28451e-22 rr_new=2.65847e-29 Az_z=2.14717
4 iter 2 rel_res=7.32151e-34 rr_new=2.68555e-53 Az_z=9.80044e-25
5 Converged at iter 2 rel_res=7.32151e-34
6 Finished iterations: 2
7 Total time = 71.4576 sec
8 RMSE = 2.13163e-14
9 --- 2 ---
10 iter 0 rel_res=4.08251e-11 rr_new=8.34998e-08 Az_z=1.84322e+24
11 iter 1 rel_res=6.31857e-26 rr_new=2.00018e-37 Az_z=0.00307822
12 Converged at iter 1 rel_res=6.31857e-26
13 Finished iterations: 1
14 Total time = 37.9174 sec
15 RMSE = 1.15463e-14
16 --- 4 ---
17 iter 0 rel_res=1.10753e-10 rr_new=6.14531e-07 Az_z=1.84322e+24
18 iter 1 rel_res=1.7692e-24 rr_new=1.56814e-34 Az_z=0.0226547
19 Converged at iter 1 rel_res=1.7692e-24
20 Finished iterations: 1
21 Total time = 22.6037 sec
22 RMSE = 2.22045e-14
23 --- 8 ---
24 iter 0 rel_res=3.07199e-11 rr_new=4.72792e-08 Az_z=1.84322e+24
25 iter 1 rel_res=5.74415e-26 rr_new=1.65304e-37 Az_z=0.00174295
26 Converged at iter 1 rel_res=5.74415e-26
27 Finished iterations: 1
28 Total time = 14.207 sec
29 RMSE = 2.44249e-14
30 --- 16 ---
31 iter 0 rel_res=6.87155e-11 rr_new=2.3656e-07 Az_z=1.84322e+24
32 iter 1 rel_res=2.59636e-24 rr_new=3.37722e-34 Az_z=0.00872077
33 Converged at iter 1 rel_res=2.59636e-24
34 Finished iterations: 1
35 Total time = 14.5603 sec
36 RMSE = 4.86278e-14
```

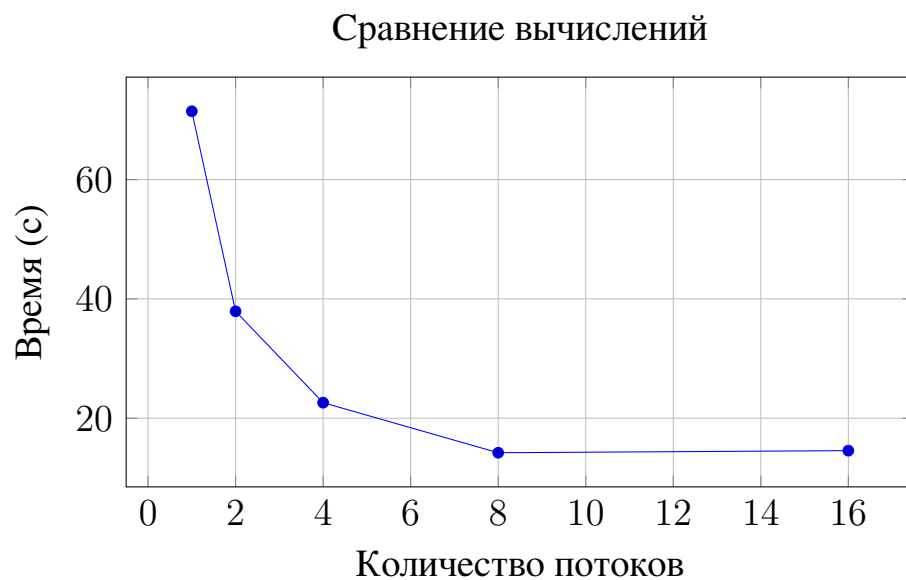


Рис. 1 — Сравнение времени выполнения с разным количеством процессорных ядер

4 Характеристики устройства

Имя устройства Acer Extensa 215-54

Оперативная память 16,0 ГБ

Процессор устройства Intel Core i5-1135G7 (2.40 ГГц, 4 ядра, 15 Вт)